

# Reinforcement Learning

## Planning and Learning with Tabular Methods

Sutton/Barto\* Chapter 8

Michael Hahsler, SMU

With figures from Sutton/Barto\*

\*Sutton and Barto, Reinforcement Learning: An Introduction,  
2<sup>nd</sup> edition, MIT Press, Cambridge, MA, 2018



# Topics of this Course

- Introduction to reinforcement learning
- Markov decision processes
- **Part I: Tabular Methods**
  - Dynamic programming
  - Monte Carlo methods
  - Temporal-difference learning
  - Multi-step bootstrapping
  - **Planning and learning with tabular methods**
- Part II: Approximate Solution Methods
  - Prediction and Control using Approximation
  - Eligibility Traces
  - Policy Gradient Methods
- Part III: Modern RL Methods
  - Deep Reinforcement Learning
  - Current Applications

# Summary of Notation

## General

|                                |                                                                             |
|--------------------------------|-----------------------------------------------------------------------------|
| $X$                            | capital letters: random variables                                           |
| $x, p$                         | lower-case letters: realizations of random variables or scalar functions    |
| $\mathbf{w}$                   | Bold lower-case letters: real-valued vectors (even if random variables)     |
| $\mathbf{W}$                   | bold capitals: matrices                                                     |
| $\alpha$                       | Greek letters: parameters (vectors if in bold)                              |
| $\Pr\{X = x\}$                 | probability that a random variable $X$ takes on the value $x$               |
| $X \sim p$                     | random variable $X$ selected from distribution $p(x) = \Pr\{X = x\}$        |
| $\mathbb{E}[X]$                | expectation of a random variable $X$ , i.e., $\mathbb{E}[X] = \sum_x p(x)x$ |
| $\operatorname{argmax}_a f(a)$ | a value of action $a$ at which $f(a)$ takes its maximal value               |

## Value Function

|               |                                                                  |
|---------------|------------------------------------------------------------------|
| $G_t$         | return (cumulative reward) following time $t$                    |
| $G_{t:h}$     | return from $t$ to $h$ (discounted and corrected)                |
| $v_\pi(s)$    | value of state $s$ under policy $\pi$ (expected return)          |
| $v_*(s)$      | value of state $s$ under the optimal policy                      |
| $q_\pi(s, a)$ | value of taking action $a$ in state $s$ under policy $\pi$       |
| $q_*(s, a)$   | value of taking action $a$ in state $s$ under the optimal policy |
| $V, V_t$      | array estimates of state-value function $v_\pi$ or $v_*$         |
| $Q, Q_t$      | array estimates of action-value function $q_\pi$ or $q_*$        |

## MDP

|                   |                                                                                                       |
|-------------------|-------------------------------------------------------------------------------------------------------|
| $s, s'$           | states                                                                                                |
| $a$               | an action                                                                                             |
| $r$               | a reward                                                                                              |
| $\mathcal{S}$     | set of all (nonterminal) states, $\mathcal{S}^+$ are all states                                       |
| $\mathcal{A}(s)$  | set of all actions available in state $s$                                                             |
| $\gamma$          | discount-rate parameter                                                                               |
| $t$               | discrete time step                                                                                    |
| $T$               | final time step of an episode (a.k.a. horizon)                                                        |
| $A_t$             | random variable for the action at time $t$                                                            |
| $S_t$             | random variable for the state at time $t$                                                             |
| $R_t$             | random variable for the reward at time $t$                                                            |
| $p(s', r   s, a)$ | probability of transition to state $s'$ and receiving reward $r$ , from state $s$ taking action $a$ . |
| $p(s'   s, a)$    | probability of transition to state $s'$ from state $s$ taking action $a$ .                            |
| $r(s, a)$         | expected immediate reward from state $s$ after action $a$ .                                           |
| $r(s, a, s')$     | expected immediate reward from state $s$ to $s'$ with action $a$ .                                    |
| $\pi(a   s)$      | probability of taking action $a$ in state $s$ under stochastic policy $\pi$                           |
| $\pi(s)$          | action taken in state $s$ under deterministic policy $\pi$                                            |

## Temporal Difference Learning

|            |                                 |
|------------|---------------------------------|
| $U_t$      | target for estimate at time $t$ |
| $\delta_t$ | TD error                        |

# Planning vs. Learning

# Planning vs. Learning

## Model-based RL

- Requires knowledge of the MDP model.
- Methods: Dynamic Programming, heuristic search
- Use the known model to **plan**

## Model-free RL

- Model is unknown.
- Methods: MC, TD
- **Learn** from interactions with the environment (sampling).



+ use the MDP as a simulation model for Model-free RL

All methods do the following:

- **Policy evaluation** (prediction): Compute a value function/policy.
- **Look ahead** to future events (rewards, states, actions) and compute a backed-up value to update an approximate value function.

# What is a Model

**Model of the environment:** Anything the agent can use to predict the outcome of actions (e.g., next state and reward).

Models can be:

- **Distribution models:** Specify the complete distribution of all possible outcomes.
  - For finite MDPs this is the function  $p(s', r | s, a)$
  - Used by dynamic programming methods for planning.
- **Sample models:** Simulate the real environment by producing one outcome for an action. The outcome needs to be a valid sample from the distribution model.
  - Are typically easier to create.
  - Can be used for MC and TD methods for learning. This is called **trajectory sampling**.

**Notes:**

- The real environment can be seen as a physically represented model with sequential sampling access.
- Model-free methods often use sample models when accessing the real environment is slow, expensive or dangerous.

Can we integrate learning and planning?

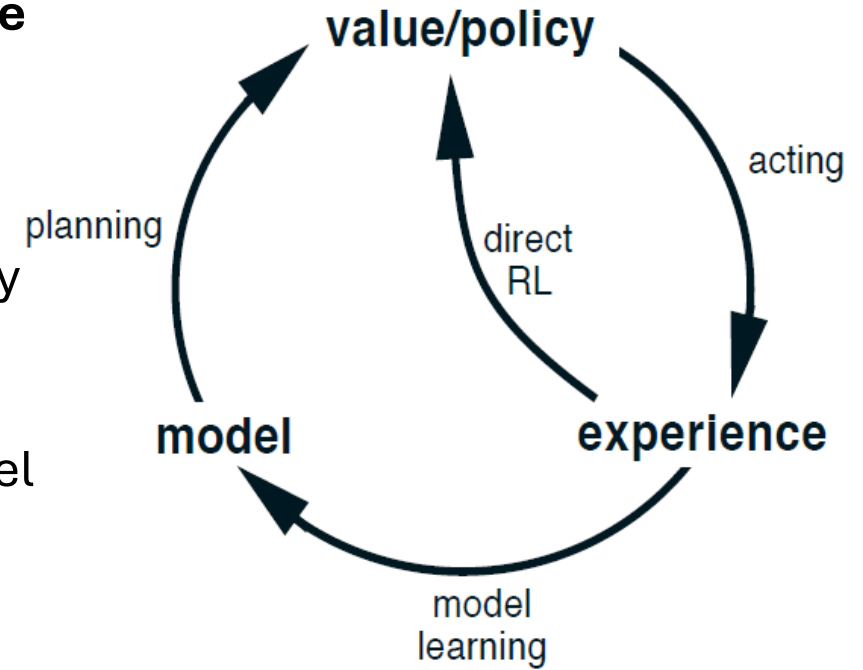
# Dyna

Integrating Planning, Acting, and Learning

# Integrating Planning, Acting, and Learning

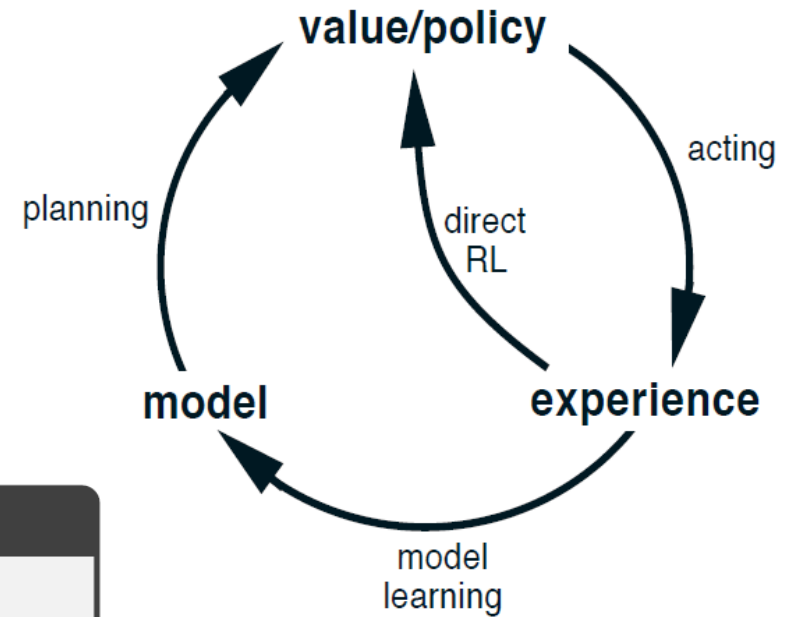
- **Direct RL is model-free Learning:**
  - Use sampled experience directly to **improve the value function/policy**. E.g., via one-step Q-learning.
- **Model Learning and Planning:**
  - Real experience is also used to **learn a model**. E.g., by recording for each  $S_t, A_t \rightarrow R_{t+1}, S_{t+1}$  to estimate  $\hat{p}(S_{t+1}, R_{t+1} | S_t, A_t)$
  - The model can be used for **planning** to improve the value function/policy. E.g., by sampling from the model to update the Q-values.

**Advantage:** Model learning and planning makes the approach much more sample efficient. Experience is not just used once but retained in the model.





# Alg. Tabular Dyna-Q



## Tabular Dyna-Q

Initialize  $Q(s, a)$  and  $Model(s, a)$  for all  $s \in \mathcal{S}$  and  $a \in \mathcal{A}(s)$

Loop forever:

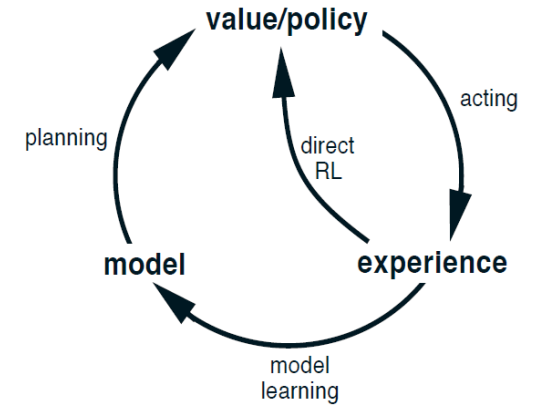
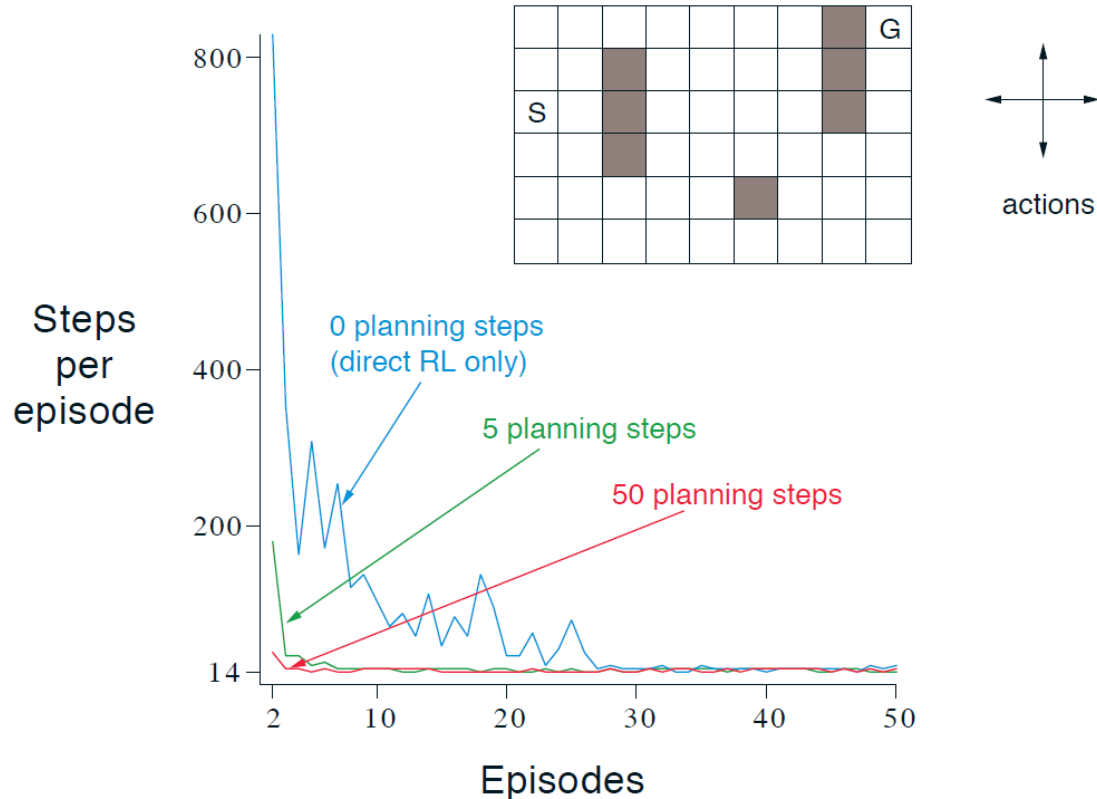
- (a)  $S \leftarrow$  current (nonterminal) state
- (b)  $A \leftarrow \varepsilon$ -greedy( $S, Q$ )
- (c) Take action  $A$ ; observe resultant reward,  $R$ , and state,  $S'$
- (d)  $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$
- (e)  $Model(S, A) \leftarrow R, S'$  (assuming deterministic environment)
- (f) Loop repeat  $n$  times:
  - $S \leftarrow$  random previously observed state
  - $A \leftarrow$  random action previously taken in  $S$
  - $R, S' \leftarrow Model(S, A)$
  - $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

Direct RL: Q-Learning

Model learning: learn  $p(s', r|s, a)$

Improve Q using the model (Planning)

# The Effect of Planning



- **Benefit:** Adding planning speeds learning by reusing sampled experience.
- **Issues:** Planning is a problem if the model is wrong:
  - Based on an insufficient number of samples.
  - $p$ -function approximation is not great.
  - Environment changes.
- Luckily, Dyna quickly corrects overly optimistic predictions since they will be selected for the next real action in the direct RL step.
- For changing environments:
  - Keep exploring.
  - Keep track of how out-of-date estimates are. I.e., for how long each action/state combination was not tried.

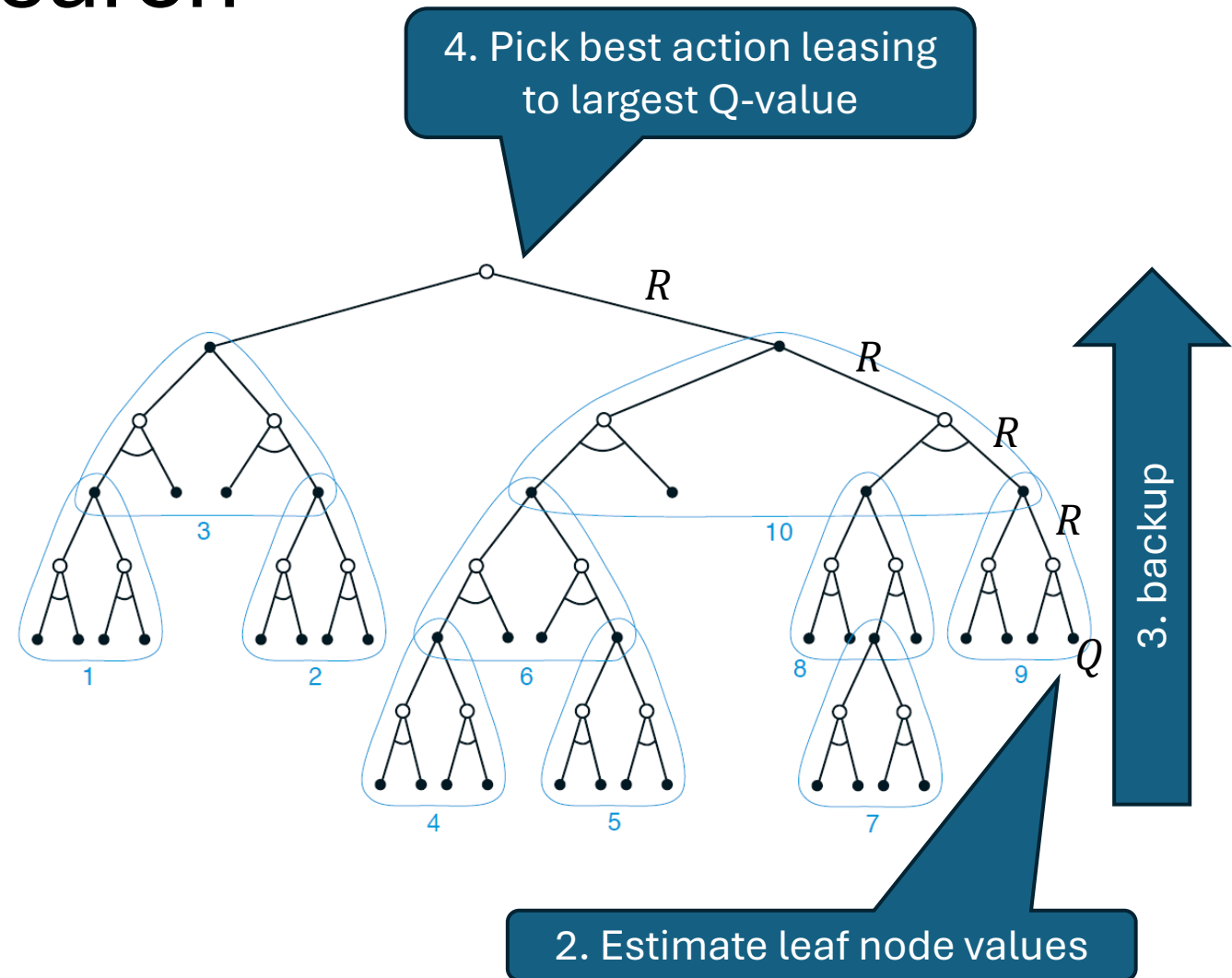
# Heuristic Search and Rollout Algorithms

Decision-time planning

# Option 1: Heuristic Search

At decision time in state  $s$ :

1. Consider a large tree of possible continuations (trajectories of a given length) for the current state.
2. Apply an approx. value function to the states in leaf nodes.
3. Back values towards the current state (using max). Similar to  $n$ -step estimation, but it always uses the best and not the observed action.
4. Choose the best next action for the current state.



**Note:** This is like heuristic (cut-off) min-max search for games.

# Option 2: Rollout Algorithms

Start with an initial policy  $\pi$  called the base policy.

At decision time in state  $s$ :

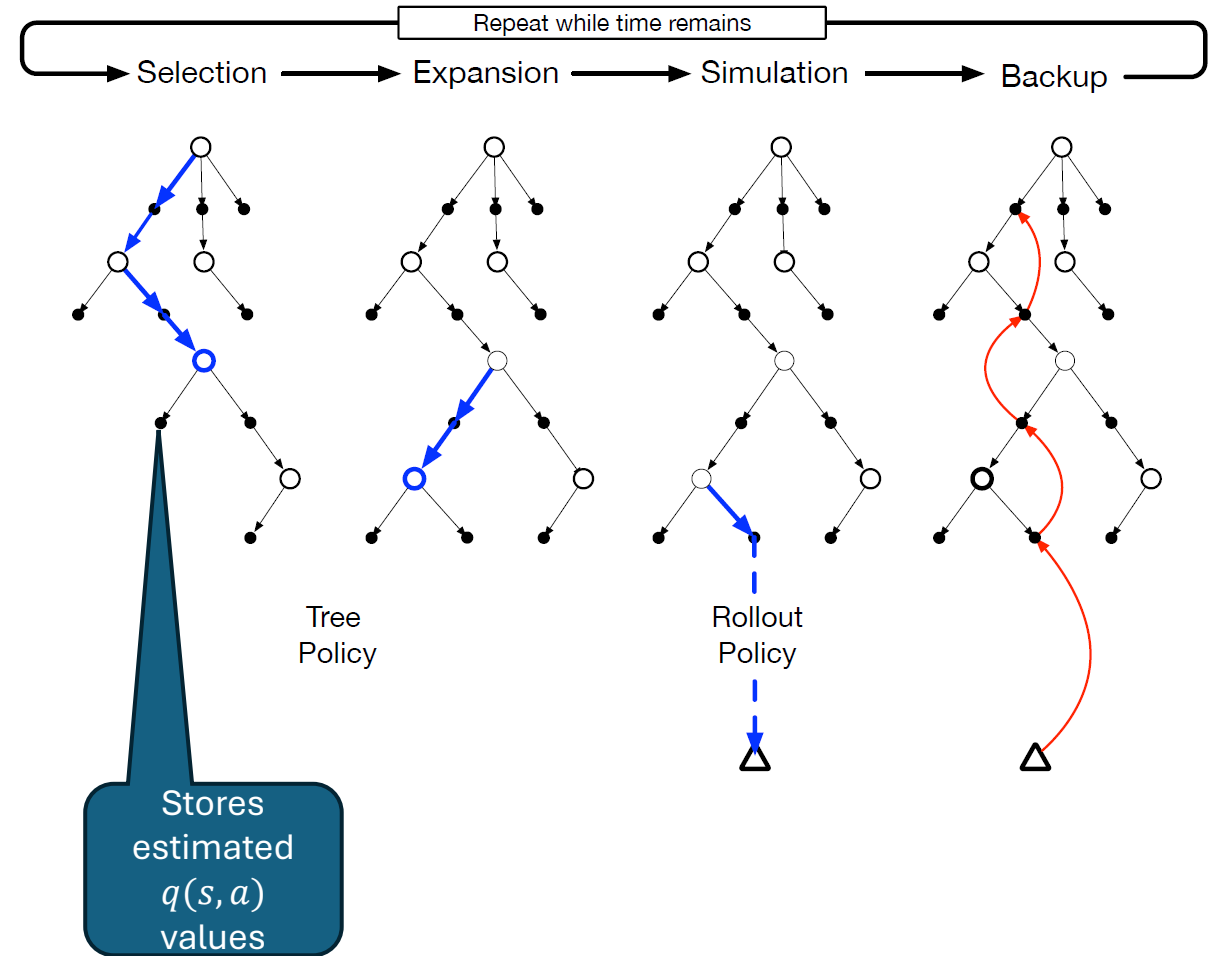
1. **Monte Carlo Policy Estimation:** Use many simulated trajectories to approximate all state/action values  $q_\pi(s, a)$  for the current state  $s$ .
2. **Choose the best action**  $a = \underset{a}{\operatorname{argmax}} q_\pi(s, a)$  according to the estimates. This is called the **rollout policy**.

## Properties:

- If the base policy is reasonable (i.e., somewhat similar to the optimal policy), rollouts are locally optimal.
- Rollout policy is guaranteed to be at least as good as the base policy (in expectation).
- **You get policy improvement without solving Bellman equations.**

# Rollout Algorithm: Monte Carlo Tree Search

- MCTS is a rollout algorithm that stores some intermediate estimates in a depth-limited tree structure.
  - **Selection:** Pick the currently best leaf node (e.g., greedy, UCB).
  - **Expansion:** Add new nodes, but keep the tree small enough.
  - **Simulation:** From a selected node, simulate complete episode(s).
  - **Backup:** update the tree with the result of the playout.
- **Advantage:** selection steers the simulation towards better trajectories. This avoids approximating the full value function.



MCTS can be seen as an RL method that selectively approximates q-values useful for choosing good actions.



# What you Should Know

- We can have different models of the environment.
  - A full **distribution model**
  - A **sample model**
- DP (planning) requires a distribution model because it uses expected updates.
- Model-free RL, like MC and TD, can use a sample model to perform sample updates.
- Decision-time planning can also use a sample model. E.g., rollout algorithms sample trajectories.
- Planning and learning are related and try to estimate the same value function.
  - **Learning:** Direct RL directly learns a value function
  - **Planning:** We learn a model used to estimate the value function.

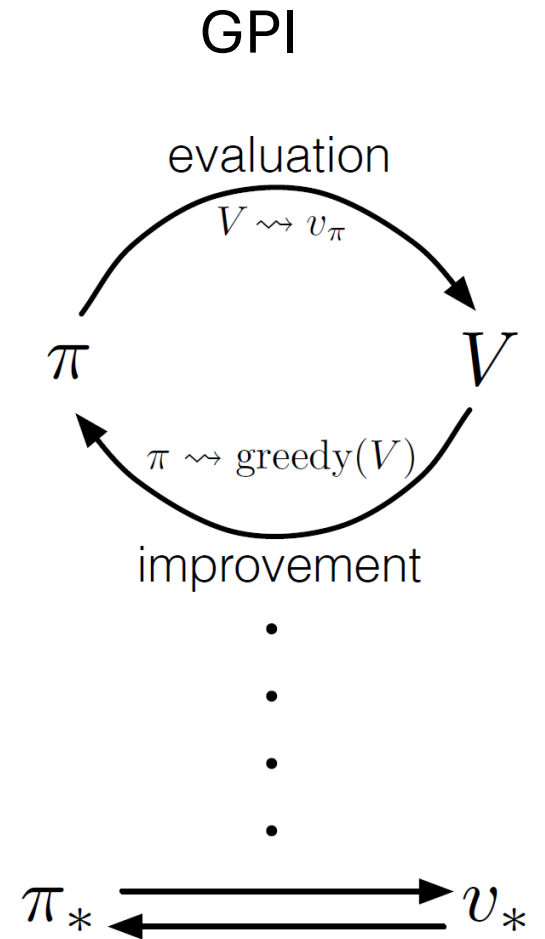
# Summary of Part I: Tabular Solution Methods



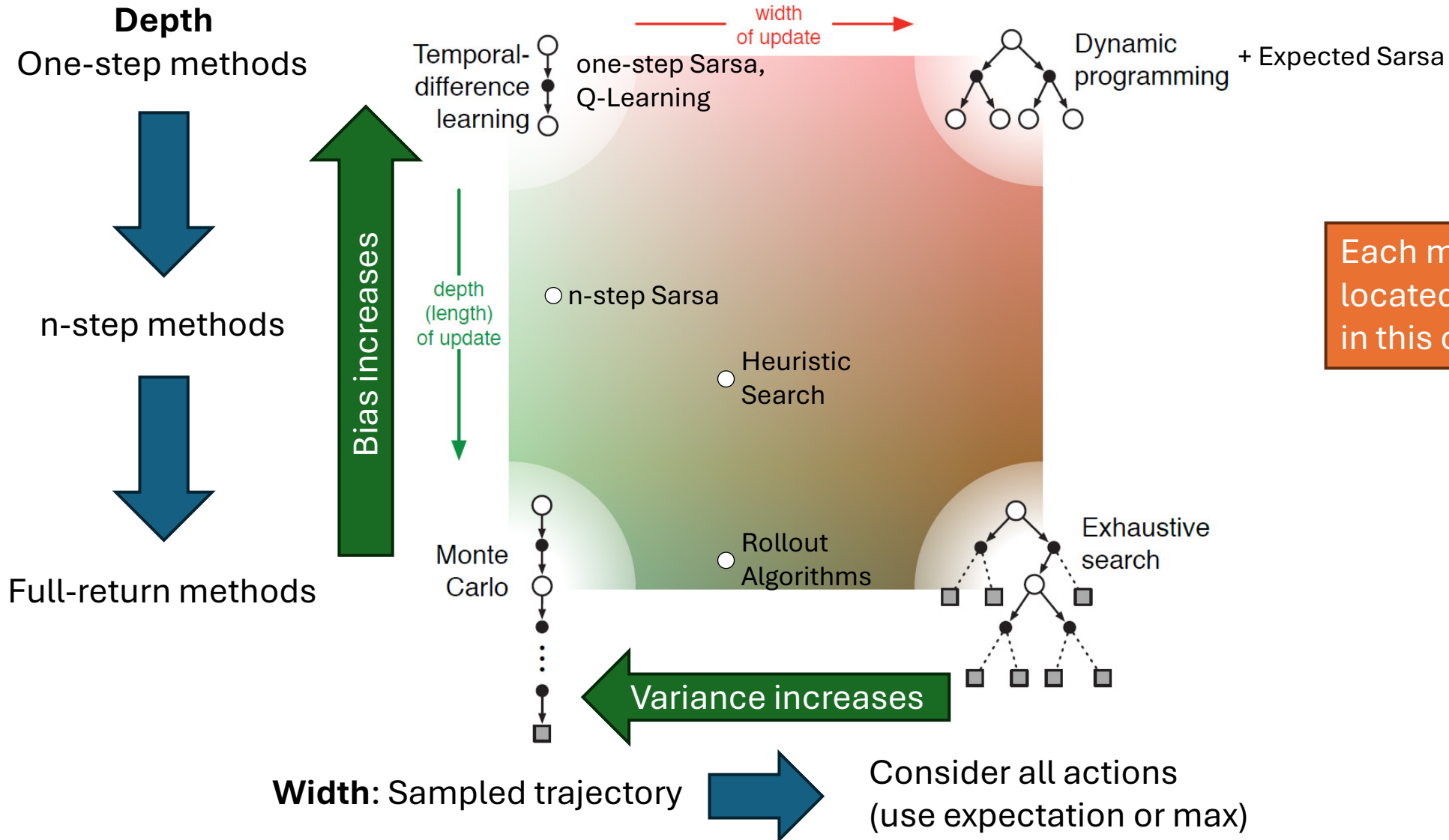
# Common Ideas

1. **Policy Evaluation:** Estimate the value function by updates based on **backing up** values along actual or possible state trajectories.
2. **Generalized policy iteration (GPI):** maintain an approx. value function and an approx. policy. Each is used to update the other. Converges due to the policy improvement theorem.

**Differences:** How to perform policy evaluation.



# Policy Evaluation: Update Depth vs. Update Width



Each method is located somewhere in this continuum.

# On-Policy vs. Off-Policy

**On-Policy:** Evaluate or learn a policy that is used to sample data.

**Off-Policy:** Evaluate or learn a target policy different from the behavior policy.

**Off-Policy is useful because:**

- **Learn optimal policies** while using exploring behavior.
- **Reuse of data is possible:** offline, and batch RL on existing training data leads to better sample efficiency.

**Off-Policy learning is harder because:**

- **Requires distributional correction:** E.g., importance sampling which is not able to use all data when the behavioral policy is very different from the target policy.
- May suffer from **high variance** caused by importance sampling.
- **Instability with non-linear function approximation:** This is a major issue for deep RL

# Other Dimensions

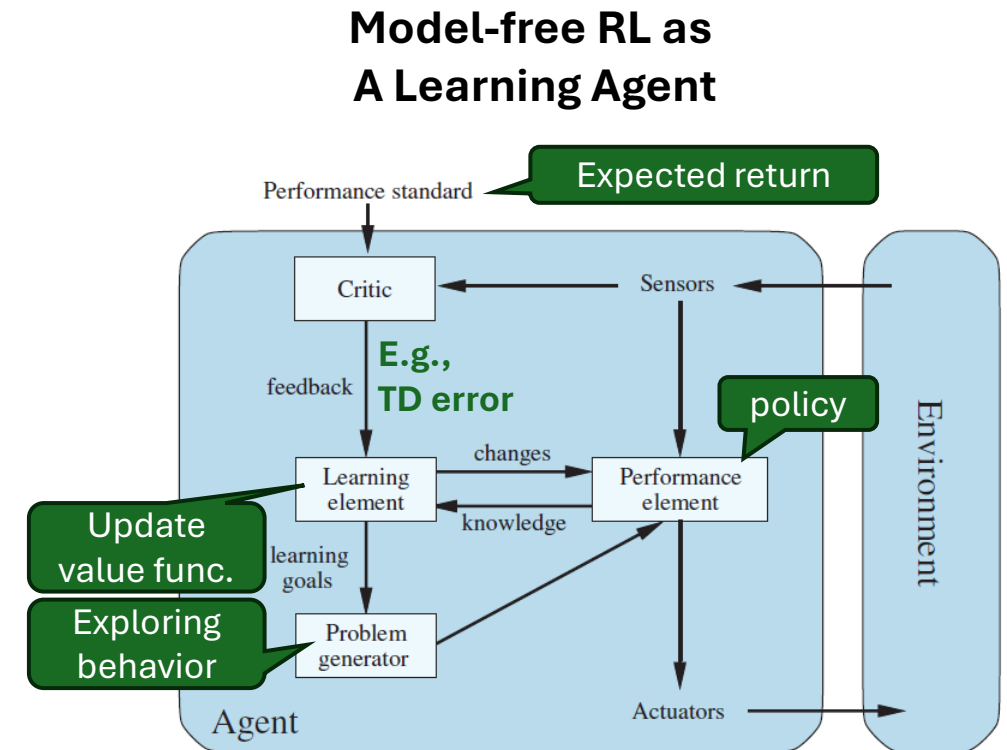
- **Definition of return:**
  - Episodic vs. continuing task
  - Discounted vs. undiscounted
- **Action values  $q$  vs. state values  $v$**
- **Exploration vs. exploitation:** This is a trade-off. Many methods need exploration (covering behavior policies) for convergence. This is related to on s. off-policy learning.
- **Synchronous vs. asynchronous updates:** Are all states updated at once or one by one in some order?
- **Real vs. simulated experience:** Are real experiences, simulations, or both used for updates?

# Relationship With Reflex Agents

- Tabular methods create a policy that defines the action for each state.
- We can implement this policy by a set of rules, one for each state.
- The agent is **learning** because it improves the policy over time.
- For episodic tasks, tabular RL implements a **learning reflex agent**.
- For continuous tasks, where the rewards express utility, we get a **learning utility-based agent**.

## Cases:

- a) **Fully observable case:**
- DP uses the MDP for **planning** to create the optimal simple reflex agent.
  - Model-free RL **learns** a simple reflex agent.
- b) **Partially observable case:** The agent constantly updates its agent state to approximate the environment state. The agent stores the agent state and uses it to decide on actions. It learns a **model-based reflex agent**.



Source: Artificial Intelligence: A Modern Approach, Chapter 2

# Issues with Tabular Methods

1. Tables need a lot of **space**. A complete table with  $Q$ -values has size  $O(|\mathcal{S}| \times |\mathcal{A}|)$   
The state space is typically extremely large, or even infinite, in continuous spaces.
2. Two states may be very similar and therefore should have similar  $Q$ -values. Tabular methods **cannot generalize across states** since each table entry is independent. We **need a lot of data** to estimate all entries.

In the next part of this course, we will talk about **approximate solution methods** to deal with these issues.